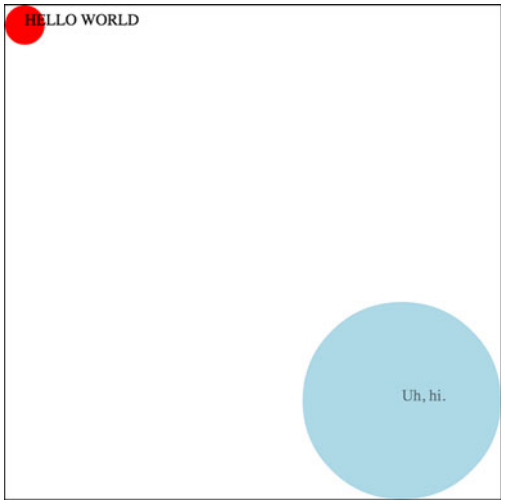


Figure 1.33 Transition behavior when associated with a delay results in a pause before the application of the attribute or style.

```
d3.select("#a").transition().delay(1000).style("opacity", 1);
d3.select("#b").transition().delay(3000).style("opacity", .75);
```

Congratulations! You've made your first dynamic data visualization. The `.transition()` method indicates that you don't want your change to be instantaneous. By chaining it with the `.delay()` method, you indicate how many milliseconds to wait before implementing the style or attribute changes that appear in the chain after that `.delay()` setting.

We'll get a bit more ambitious later on, but before we finish here, let's look at another `.transition()` setting. You can set a `.delay()` before applying the new style or attribute, but you can also set a `.duration()` over which the change is applied. The results in your browser should move the shapes in the direction of the arrows in figure 1.34:

```
d3.selectAll("circle").transition().duration(2000).attr("cy", 200);
```

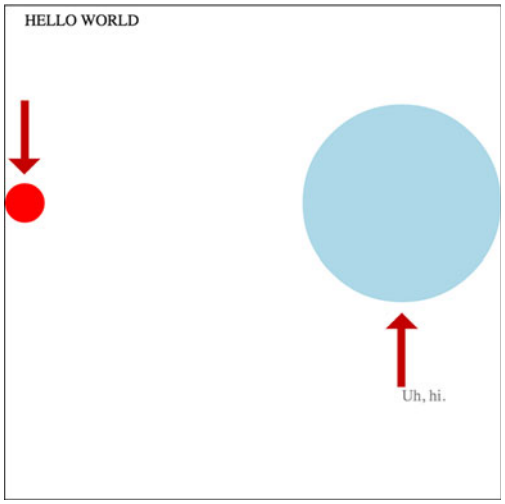


Figure 1.34 Transition behavior when associated with position makes the shape graphically move to its new position over the course of the assigned duration. Because you used the same y position for both circles, the first circle moves down and the second circle moves up to the y position you set, which is between the two circles.